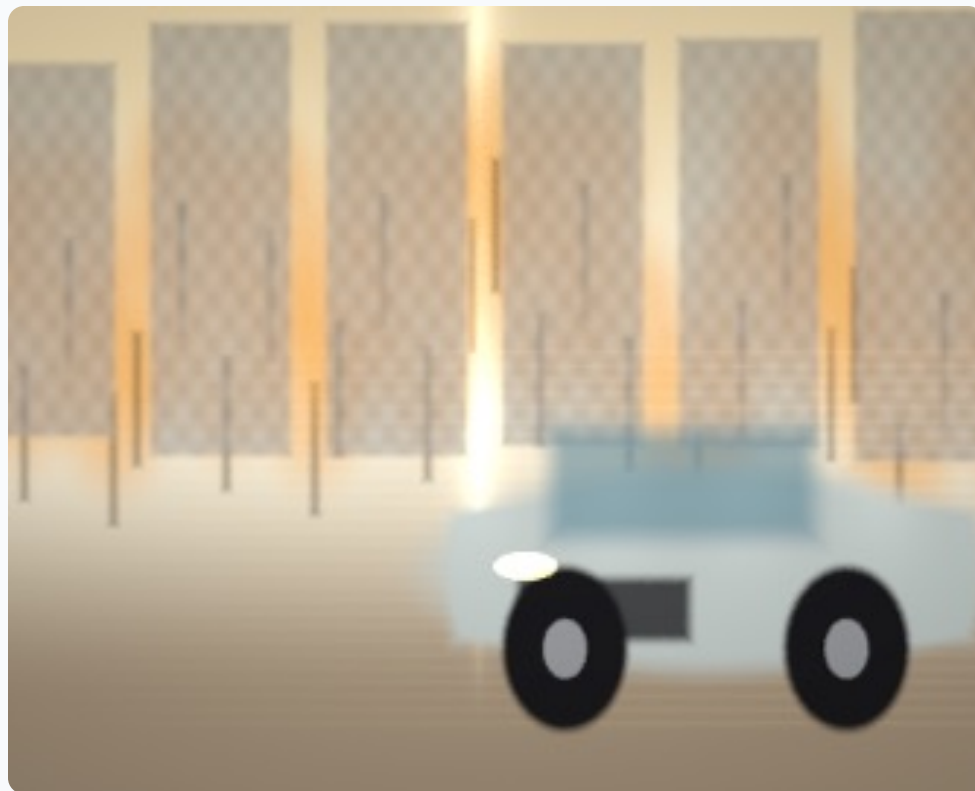


光线追踪渲染器的并行优化

串行基准 OpenMP 多线程 CUDA GPU



4.02x

OpenMP 加速比

400 x 300, 64 samples, 8 threads

像素级并行

优化目标

每个像素可独立追踪光线

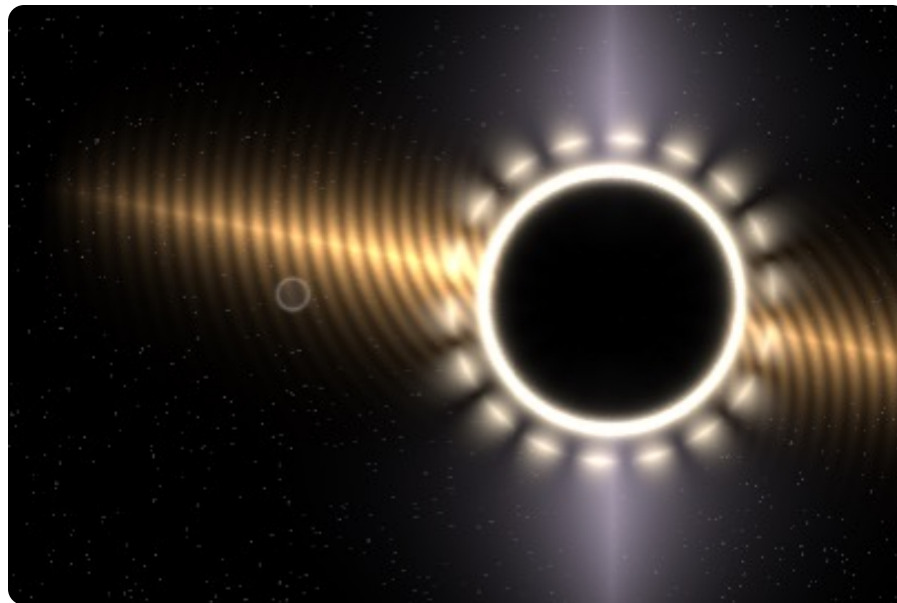
已编译提交

CUDA 状态

通过 Slurm 进入 GPU 队列

为什么选择光线追踪作为优化对象

- 光线追踪能生成真实的反射、折射、阴影和景深效果
- 每个像素需要多次采样和多轮相交计算，计算密度高
- 像素之间天然独立，适合 CPU 多线程和 GPU 大规模并行
- 实验结果可以同时体现性能提升和图形渲染效果变化



目标：用高性能计算技术优化一个可运行的渲染程序，并分析不同并行方案的收益与限制。

项目按三个版本逐步优化

V1 Serial

单线程逐像素追踪，作为性能基准



V2 OpenMP

对像素行循环并行化，利用 CPU 多核



V3 CUDA

每个 GPU thread 负责一个像素，显存保存场景数据

核心思路：保持相同渲染算法，逐步改变任务调度方式，从而对比并行化带来的性能变化

。

OpenMP 优化把独立像素分配给多个 CPU 线程

- 先将像素颜色写入 framebuffer，避免多线程同时写文件
- 使用 parallel for 并行图像行循环
- 使用 dynamic 调度缓解不同像素追踪深度不一致的问题
- 通过 OMP_NUM_THREADS 控制线程数，便于做扩展性分析

```
#pragma omp parallel for schedule(dynamic, 1)
for (int j = 0; j < HEIGHT; j++) {
    for (int i = 0; i < WIDTH; i++) {
        framebuffer[row * WIDTH + i] = color;
    }
}
```

```
webssh 1 x
Done.
Render time: 39.1013 s
Render time: 39.1013 s
(base) [bvtui@login03 1]$ cat results/benchmark_results.csv
cat: results/benchmark_results.csv: No such file or directory
(base) [bvtui@login03 1]$ ls -lh results/
total 2.6M
-rw-r--r-- 1 bvtui syren_group 1.3M Jun 16 16:52 v1_output.ppm
-rw-r--r-- 1 bvtui syren_group 1.3M Jun 16 16:55 v2_output.ppm
(base) [bvtui@login03 1]$ head -n results/v1_output.ppm
head: invalid number of lines: 'results/v1_output.ppm'
(base) [bvtui@login03 1]$ head -n 5 results/v1_output.ppm
P3
400 300
255
220 235 255
220 235 255
(base) [bvtui@login03 1]$
(base) [bvtui@login03 1]$
(base) [bvtui@login03 1]$
(base) [bvtui@login03 1]$ g++ -O2 -std=c++17 -fopenmp v2_openmp/main.cpp -o v2_openmp/v2_openmp
(base) [bvtui@login03 1]$ OMP_NUM_THREADS=8 ./v2_openmp/v2_openmp --width 400 --height 300 --samples 64
Threads=8 Render time: 9.72383 s
Threads=8 Render time: 9.72383 s
(base) [bvtui@login03 1]$
```

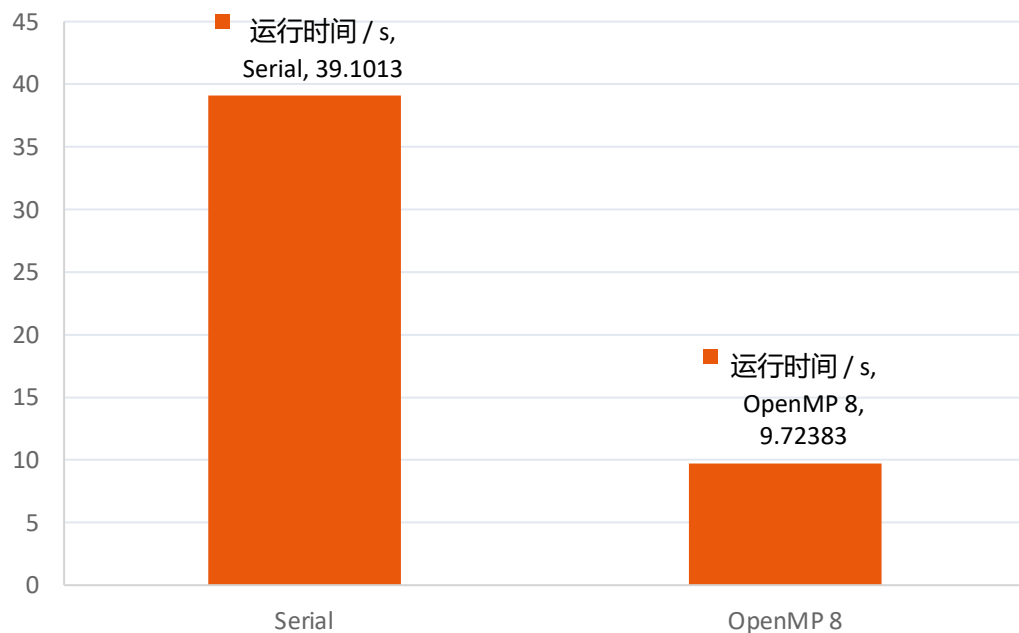
CUDA 版本把像素计算映射到 GPU 线程

- 16 x 16 线程块组织二维网格
- 每个线程独立生成随机采样并计算一个像素颜色
- 递归光线追踪改为迭代流程，降低 GPU 栈压力
- 球体与材质通过 cudaMalloc 放入显存，作为 kernel 参数传入

```
module load intel/cuda/12.1
nvcc -O3 -arch=sm_80 v3_cuda/main.cu -o
v3_cuda/v3_cuda
sbatch scripts/submit_gpu_slurm.sh
```

**登录节点不直接跑 CUDA 程序；正式测试
通过 Slurm 提交到 gpu_4090 计算节点。**

已有超算结果显示 OpenMP 获得明显加速



39.1013 s

串行版本

400 x 300, 64 samples

9.72383 s

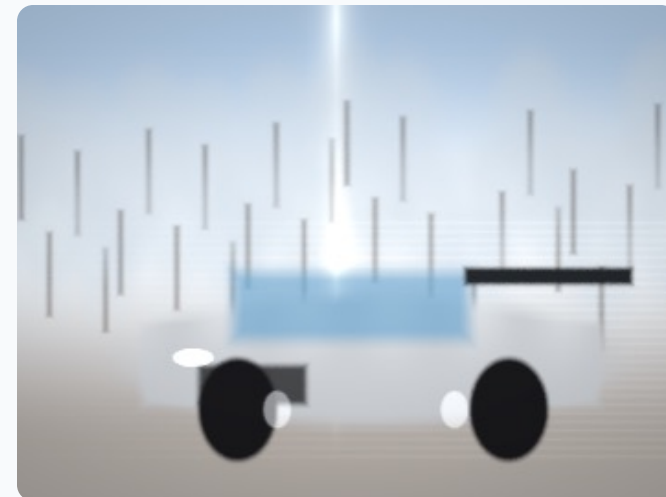
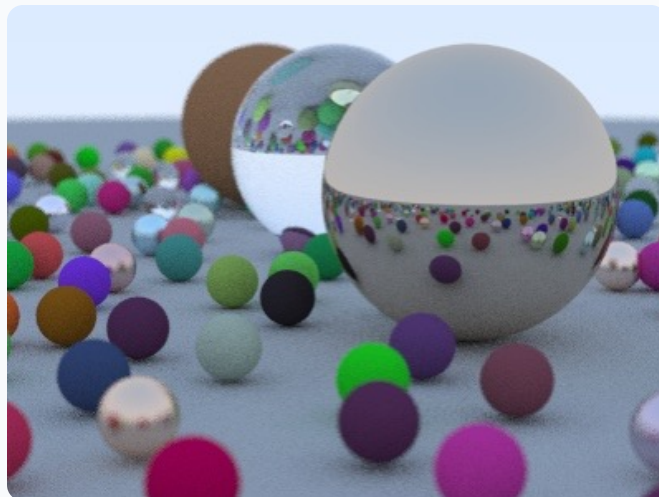
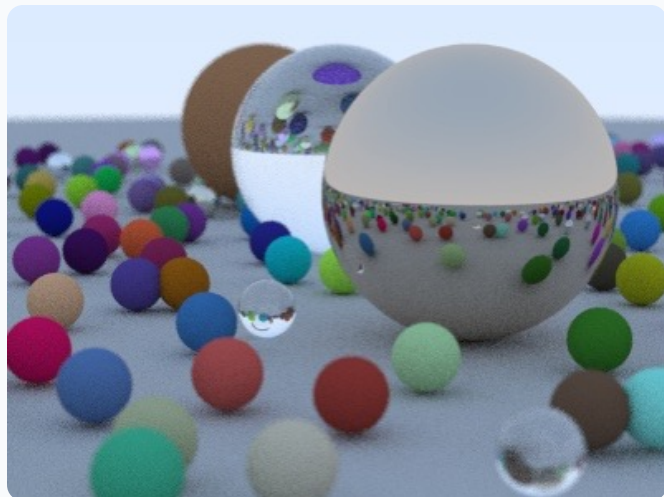
OpenMP 版本

8 CPU threads

加速比 = $39.1013 / 9.72383 \approx 4.02x$, 并行效率约 50.3%。

结果展示

渲染结果覆盖基础验证和游戏化展示场景



PPM 输出可直接在超算生成；报告中已转为 JPG/PNG 便于展示。扩展场景使用程序生成，不依赖商业游戏素材。

课程设计已经形成可复现的提交包

- 完成串行、OpenMP、CUDA 三个版本的源码与运行说明
- 完成超算 CPU/OpenMP 实测，OpenMP 8 线程加速约 4.02 倍
- CUDA 已解决编译问题，并按平台要求通过 Slurm 提交 GPU 队列
- 提交材料包含源代码、实验报告、答辩 PPT PDF 和结果图片

后续可继续补充 CUDA 最终运行时间、OBJ 模型加载和 BVH 加速结构。